



Original article

Deep reinforcement learning-based trajectory planning with continuous pose representation for 6-DoF free-floating space robot

Yuhui Hu ^{ID}, Dong Zhou ^{ID,*}, Weiran Yao, Xiangyu Shao ^{ID}, Guanghui Sun

School of Astronautics, Harbin Institute of Technology, Harbin, 150001, China

ARTICLE INFO

Communicated by Jérôme Morio

Keywords:

Space manipulator
Reinforcement learning
Continuous pose representation
Hindsight experience replay
Curriculum learning

ABSTRACT

Trajectory planning for free-floating space manipulator is a challenging task due to the complex coupling between the manipulator and its floating base. In this paper, we propose the RTPC algorithm, a reinforcement learning-based trajectory planning method to address the 6-DoF free-floating space manipulation tasks. To overcome the discontinuity and non-uniqueness of conventional pose representations, we introduce a continuous formulation based on transformation matrix and incorporate it into the state space, enabling more stable learning and improving trajectory planning performance. To address the challenges of efficient convergence and enhance exploration in the early and mid-stages of training, we designed the Hybrid Efficient Learning (HybridEL) framework, which combines hindsight experience replay with precision-based curriculum learning. We conduct comprehensive experiments to validate the effectiveness of our proposed algorithm. The results demonstrate that our approach achieves better convergence and superior performance in space manipulator trajectory planning tasks compared to existing methods, while maintaining real-time capability. The code and simulation configuration for this work are publicly available at <https://github.com/HIT-YuhuiHu/RTPC>.

1. Introduction

With the increasing demand for space exploration, space manipulators have garnered widespread attention due to their advantages of safety and efficiency. It has been instrumental in tasks such as on-orbit satellite servicing [1], space station assembly [2], and space debris removal [3]. Space manipulator systems generally composed of a base spacecraft and at least one robotic manipulator. Based on the control state of the spacecraft, space manipulators can be categorized into free-flying and free-floating modes [4].

In the free-flying mode, the spacecraft's pose is fully controllable, eliminating the dynamic coupling between the base and the manipulator, thereby simplifying the trajectory planning task for space manipulator. This mode typically requires the spacecraft to adjust its pose in real-time [5], which inevitably leads to high fuel consumption and consequently shortens the spacecraft's lifespan. A more economical method is free-floating, in which only the manipulator is controlled. However, in the space environment, unavoidable dynamic coupling occurs between the manipulator and the base spacecraft. Some researchers have introduced the generalized Jacobian matrix (GJM) [4], which incorporates

velocity of the base to achieve a more precise control model for the manipulator. Nevertheless, due to the time-varying nature of the GJM during manipulator motion, these methods involve substantial computational overhead.

Other researchers have treated the coupling between the free-floating base and the manipulator as a constraint in trajectory planning, solving the problem through optimization methods. These approaches can be further classified into indirect and direct methods [6]. Indirect methods convert trajectory planning into a boundary value problem, which is subsequently solved using variational methods and numerical techniques. Ratajczak et al. [7] applied the Lagrange Jacobian motion planning algorithm for trajectory planning of 3 or 4 DoF free-floating manipulator. Although indirect methods offer theoretically optimal solutions, these are typically limited to low-dimensional optimization tasks [8]. In contrast, direct methods often utilized advanced optimization algorithms [9–11], which have made significant contributions to trajectory planning. However, these optimization-based methods face challenges in high-dimensional, non-convex, and dynamic environments. Additionally, they can be computationally expensive, sensitive to initial conditions, and prone to converging to local optima.

* Corresponding author.

E-mail addresses: huyuhui@hit.edu.cn (Y. Hu), dongzhou@hit.edu.cn (D. Zhou), yaoweiran@hit.edu.cn (W. Yao), xiangyushao@hit.edu.cn (X. Shao), guanghuisun@hit.edu.cn (G. Sun).

<https://doi.org/10.1016/j.ast.2025.110540>

Received 29 December 2024; Received in revised form 9 June 2025; Accepted 24 June 2025

Available online 3 July 2025

1270-9638/© 2025 Elsevier Masson SAS. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

In recent years, the emergence of deep reinforcement learning (DRL) algorithms led to the development of numerous model-free trajectory planning methods [12,13]. These approaches, which do not rely on precise system models, offer innovative strategies to address the challenges posed by space manipulator trajectory planning tasks. Initially, DRL algorithms were chosen to compensate for model inaccuracies, enabling systems to better handle disturbances [14]. As more advanced algorithms were developed, researchers started to employ DRL as a controller for simple space manipulator system [15]. Over time, this approach was extended to achieve effective end-effector position control in increasingly complex scenarios [16–18].

As research progressed, incorporating space manipulator orientation into trajectory planning tasks garnered significant attention, leading to substantial advancements [19,20]. These approaches commonly rely on off-policy DRL algorithms to ensure convergence in high-dimensional spaces. However, this comes at the cost of reduced exploration, ultimately leading to limited precision. Additionally, the pose representation becomes another key difficulty in achieving high accuracy. Commonly used representations, such as Euler angles [21,22] and quaternions [23,24], have been applied with varying levels of success. Nevertheless, the discontinuity of pose representations, combined with sparse and delayed rewards, often leads to unstable training and limited performance in DRL-based space manipulator applications.

To overcome these challenges, we propose a DRL-based trajectory planning algorithm for 6-DoF free-floating space manipulator that enables the end-effector to reach target positions and orientations. Specifically, we employ an on-policy DRL algorithm to enhance exploration in high-dimensional spaces. By designing a continuous pose representation, our method significantly reduces orientation errors. Furthermore, we design a hybrid efficient learning framework to improve the learning efficiency and enable DRL algorithms to achieve high trajectory planning accuracy for space manipulator. The main contributions of this work are summarized as follows:

- 1) We propose the **RTPC** (Reinforcement learning-based Trajectory Planning with Continuous pose representation) algorithm for trajectory planning of a 6-DoF free-floating space manipulator. By introducing a continuous pose representation from the transformation matrix, our method achieves higher precision in trajectory planning tasks.
- 2) We develop a Hybrid Efficient Learning framework, **HybridEL**, integrating hindsight experience replay and precision-based curriculum learning to ensure the convergence of on-policy DRL algorithms.
- 3) Extensive comparative and ablation experiments validate the proposed algorithm's performance and demonstrate the role of each module in improving DRL convergence. Comprehensive robustness evaluations under challenging conditions, such as spacecraft spin and observation delay, confirm the effectiveness and stability of the RTPC algorithm in handling space-specific disturbances.

The rest of this article is organized as follows. Section 2 provides an overview of the related work. Section 3 presents the model of free-floating space manipulator and formulates the Markov Decision Process. Section 4 introduces our proposed method, including the RTPC and the HybridEL framework. Section 5 provides extensive experimental results to validate the effectiveness and superiority of the proposed algorithm. Finally, Section 6 concludes this article.

2. Related work

Model-based trajectory planning methods were initially applied to space manipulator systems. Yoshida et al. [25] introduced the Zero Reaction Maneuver (ZRM) method, leveraging the redundant DoF of space robotic manipulator for zero-reaction control. However, this approach introduces additional singularities and limits the manipulator's workspace. Several researchers have investigated various strategies to

mitigate singularities encountered during the planning process [26–28]. Meanwhile, some works have turned to optimization-based algorithm to reduce dependence on precise models. Approaches such as Particle Swarm Optimization (PSO) [9], Differential Evolution (DE) [10], and Chaotic PSO (CPSO) [11] have been applied for efficient and precise trajectory planning. While effective for fixed start and end positions, they perform poorly under significant disturbances.

With the continued advancement of DRL, a variety of model-free algorithms have been successfully applied across multiple domains, including Go [29], computer games [30], and robotic systems [31]. DRL guides an agent's interactions by a reward function, enabling agent to learn an optimal strategy by maximizing cumulative rewards, instead of relying on task-specific solutions [32]. This characteristic of achieving high performance without relying on physical models or real experiments makes DRL an ideal choice for space missions [33–35].

In the context of space robotic manipulation, Hu et al. [15] and Du et al. [16] applied the Deep Deterministic Policy Gradient (DDPG) algorithm to trajectory planning for free-floating planar multi-link space manipulators. Building on these approach, Wu et al. [18] extended DDPG to dual-arm space manipulators, achieving path planning for both fixed-base and free-floating scenarios. With further research, there has been a gradual shift in focus from position control to pose control. Li et al. [19] and Al et al. [20] applied DDPG to space manipulator trajectory planning, focusing on achieving the end-effector's pose control for free-floating systems. In this process, the representation of pose, especially orientation, has garnered significant attention. Euler angles were widely used in DRL algorithms [19,21,22] due to their intuitiveness, but were eventually replaced by more efficient quaternion representation [20,23]. Nevertheless, both representations inherently suffer from discontinuities, leading to approximation errors in DRL [36]. To overcome these limitations, we designed a continuous pose representation building upon the homogeneous transformation matrix. Experimental results indicate that this approach facilitates faster and more effective convergence in minimizing pose errors.

The aforementioned studies predominantly utilize off-policy DRL, particularly the DDPG algorithm. While these methods offer faster convergence, their limited exploration of off-policy DRL often lead to entrapment in local optima. In contrast, on-policy algorithms, such as Proximal Policy Optimization (PPO), excel in high-dimensional optimization tasks due to their superior exploration capabilities [37]. A major limitation of on-policy DRL is the insufficient exploration efficiency, especially in space manipulator trajectory planning tasks. Hindsight Experience Replay (HER) [38] has emerged as a potential solution to this problem. Through experience replay, HER converts failed samples into successful experiences, facilitating more effective training. However, because on-policy DRL algorithms can only use experiences collected by the current policy for updates, integrating HER with such algorithms seems to be impossible.

Another strategy to enhance exploration efficiency is curriculum learning (CL) [39], which entails designing a sequence of tasks that gradually evolve from simple to complex. This approach starts with easier tasks to facilitate early learning progress, progressively increasing task difficulty to develop a comprehensive model. However, applying CL to space manipulator tasks is difficult, particularly with free-floating bases where task complexity is harder to evaluate. In our work, we developed HybridEL, a Hybrid Efficient Learning framework that integrates HER and CL, which is incorporated into the PPO algorithm to enhance the convergence of trajectory planning tasks for free-floating space manipulator systems.

3. Problem statement

3.1. Modeling of free-floating space manipulator

The free-floating space manipulator system consists of a base spacecraft and a space manipulator. The base spacecraft freely floats in the

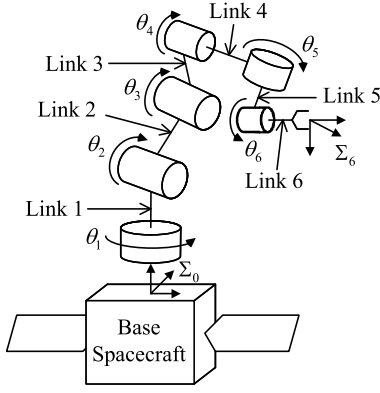


Fig. 1. Model of the 6-DoF free-floating space manipulator.

Table 1
Kinematic parameters of the free-floating space manipulator system.

Parameters	Representation
M_b	mass of the base spacecraft
m_i	masses of the manipulator's i -th link
I_b	inertia matrix of the base spacecraft
I_i	inertia matrix of the manipulator's i -th link
r_b	position vector of the base spacecraft
r_e	position vector of the end-effector
θ_i	angle of the manipulator's i -th joint

microgravity environment, with its manipulator rigidly connected. In this paper, we designed a cubic structure with solar panels to simulate the base spacecraft, and selected a 6-DoF robotic arm to serve as the space manipulator. Fig. 1 presents the schematic diagram of the system, with its kinematic parameters detailed in Table 1.

The objective of space manipulator trajectory planning is to control the manipulator's end-effector to achieve a specified pose, which includes both position and orientation. Based on momentum and angular momentum conservation, the kinematic equation of the free-floating space manipulator is derived as:

$$\dot{r}_e = -(J_b H_b^{-1} H_m - J_m) \dot{\Theta} = J^* \dot{\Theta} \quad (1)$$

where J_b and J_m are the Jacobian matrices of the base and the manipulator, respectively, and $\Theta = [\theta_1, \theta_2, \dots, \theta_6]$ represents the joint angles of the manipulator. H_b and H_m represent the coupling inertia matrices of the base and the manipulator, respectively, and J^* denotes the GJM, it is dependent on the system's inherent parameters and motion characteristics. Furthermore, the dynamic equation of the free-floating space manipulator can be described as:

$$H(\Theta) \ddot{\Theta} + C(\Theta, \dot{\Theta}) \dot{\Theta} = \tau \quad (2)$$

where $H(\Theta)$ represents the system's inertia matrix, and $C(\Theta, \dot{\Theta})$ accounts for the Coriolis force. For detailed derivations and explanations of these equations, including J^* , $H(\Theta)$ and $C(\Theta, \dot{\Theta})$, please refer to [40].

Due to the dynamic coupling between the space manipulator and the base, trajectory planning for free-floating systems exhibits more complex and time-varying nature compared to fixed-base counterparts. These issues impose major limitations on model-based algorithms, affecting both their accuracy and real-time performance. In contrast, model-free algorithms, especially DRL algorithms, provide a promising alternative by directly optimizing trajectory planning tasks without relying on an explicit model, which is the focus of this paper.

3.2. Markov decision process formulation

This paper formulates the trajectory planning task of free-floating space manipulators as a Markov Decision Process (MDP) to enable the

application of model-free DRL algorithms. The MDP is defined as a tuple $\langle S, A, P, R, \gamma \rangle$, where S and A represent the finite state and action spaces that the agent can explore and from which actions are selected, respectively. The state transition probability matrix P is defined as the probability distribution for reaching a new state $s' \in S$, given the current state $s \in S$ and action $a \in A$:

$$P := P_{s \rightarrow s'}^a = \mathbb{P}[s_{t+1} = s' | s_t = s, a_t = a] \quad (3)$$

here, R denotes the reward function, dependent on the transition between states s and s' . The discount factor $\gamma \in [0, 1]$ is introduced to balance immediate and future rewards.

The MDP of space robotic manipulation is described as follows: at a time step t , the agent observes the state s_t , it executes an action a_t based on a policy $\pi(s_t)$. The environment subsequently transitions to a new state s_{t+1} according to the state transition probability P , and generates a reward r_t immediately. The goal of RL algorithm is to find an optimal policy π^* that maximizes the expected cumulative discounted reward:

$$\pi^* = \arg \max_{\pi} E_{\pi} \left[\sum_t \gamma^t r(s_t, a_t) \right] \quad (4)$$

4. Method

In this section, we present the RTPC algorithm, which derives a continuous pose representation (CPR) from the homogeneous transformation matrix to eliminate discontinuities in orientation representation. We further integrate HER with precision-based curriculum learning into the HybridEL framework, thereby accelerating convergence of the on-policy DRL algorithm. A structural schematic of RTPC algorithm with the HybridEL framework is shown in Fig. 2.

4.1. Learning-based trajectory planning algorithm

4.1.1. PPO algorithm

As previously discussed, on-policy RL methods generally perform better in high-dimensional spaces. In this paper, we employ PPO [41] as the learning algorithm for the agent. PPO is a policy gradient method based on the actor-critic architecture, where the policy is adjusted to maximize the expected cumulative reward while ensuring stable updates. The policy network, denoted by π , outputs a probability distribution over actions, while the value function network, denoted by V , estimates the expected return from the current state. To ensure sufficient representative ability, both networks are implemented as neural networks, parameterized by θ and ϕ , respectively.

The objective of PPO is to maximize the following policy objective function:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}(\tau)} \left[\sum_t R(s_t, a_t) \right] = \mathbb{E}_{\tau \sim \pi_{\theta}(\tau)} [R(\tau)] \quad (5)$$

in which τ represents trajectories of the form $(s_1, a_1, s_2, a_2, \dots, s_T, a_T)$.

PPO works by estimating the policy gradient, and then applies a gradient ascent algorithm to optimize this gradient estimate:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}(\tau)} \left[\left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right) R(\tau) \right] \quad (6)$$

During the training process, these gradients are backpropagated through the neural network to update the parameter θ . The primary challenge in policy gradient methods is reducing the variance of gradient estimates to stabilize policy improvements. To address this issue, PPO employs the actor-critic architecture by reformulating reward signals using the advantage function:

$$Q^{\pi}(s, a) = \sum_t \mathbb{E}_{\pi_{\theta}} [R(s_t, a_t) | s, a] \quad (7)$$

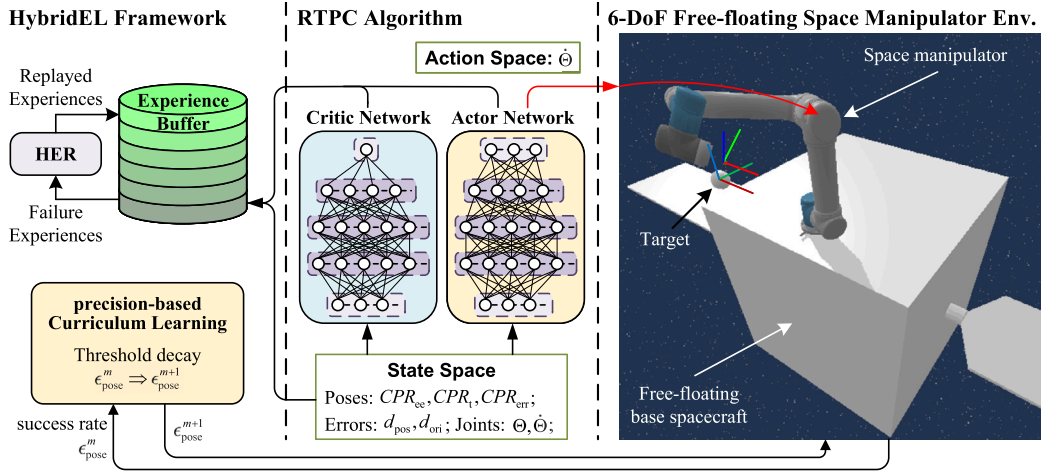


Fig. 2. Structural schematic of RTPC algorithm with HybridEL framework.

$$V^\pi(s) = \sum_t \mathbb{E}_{\pi_\theta} [R(s_t, a_t) | s] \quad (8)$$

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s) \quad (9)$$

The advantage function $A^\pi(s, a)$ is used to describe how favorable a particular action a is in a given state s . The calculation of A^π depends on the value function $V^\pi(s)$, which, cannot be obtained directly. To solve this problem, the critic network approximates the value function using reward values from the collected samples.

The actor network in PPO updates by maximizing the surrogate objective function:

$$L(\theta) = \hat{\mathbb{E}}_t [\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)] \quad (10)$$

where $\hat{A}_t$ and $\hat{\mathbb{E}}_t$ represent the empirically estimated advantage function and expectation, respectively, and $r_t(\theta)$ is the probability ratio:

$$r_t(\theta) = \frac{\pi_\theta(a_t, s_t)}{\pi_{\theta_{\text{old}}}(a_t, s_t)} \quad (11)$$

by clipping this ratio within the range $[1 - \epsilon, 1 + \epsilon]$, the similarity between the old and new policies is maintained, enhancing the stability of policy updates.

The critic network in PPO updates by minimizing the value loss, which is derived from the Temporal Difference (TD) error. The TD error quantifies the difference between the predicted value and an estimate of the cumulative reward, including immediate and future returns. The value loss is expressed as:

$$L(\phi) = \hat{\mathbb{E}}_t [(V_\phi(s_t) - R_t)^2] \quad (12)$$

where R_t represents the actual returns observed from the environment.

4.1.2. State and action representation

The selection of state and action spaces is critical to the success of DRL algorithms. For trajectory planning task, the state space should incorporate as many relevant observable parameters of the free-floating space manipulator system as possible. The pose of the end-effector and the target, which encompassed both position and orientation in 3D space, constitutes the fundamental information required for trajectory planning. While Euler angle and quaternion are widely used to represent orientation, the inherent discontinuity and non-uniqueness can hinder the convergence of DRL algorithms and reduce accuracy in space trajectory planning tasks.

To address these issues, we propose a continuous pose representation based on the homogeneous transformation matrix. Given a world coordinate system W , the pose for an object in W can be represented as H_o^W :

$$H_o^W = \begin{bmatrix} R_o^W & \vec{p}_o^W \\ 0 & 1 \end{bmatrix} \quad (13)$$

where R_o^W denotes the rotation matrix of the object. Assuming the object rotates by angles α , β , and γ relative to the X , Y , and Z axes of W , the rotation matrix can be expressed as:

$$R_o^W = \begin{bmatrix} c_\beta c_\gamma & -c_\beta s_\gamma & s_\beta \\ c_\alpha s_\gamma + s_\alpha s_\beta c_\gamma & c_\alpha c_\gamma - s_\alpha s_\beta s_\gamma & -s_\alpha c_\beta \\ s_\alpha s_\gamma - c_\alpha s_\beta c_\gamma & s_\alpha c_\gamma + c_\alpha s_\beta s_\gamma & c_\alpha c_\beta \end{bmatrix} \quad (14)$$

$$= \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} = [\vec{r}_1 \quad \vec{r}_2 \quad \vec{r}_3]$$

where $\alpha, \gamma \in (-\pi, \pi]$, $\beta \in (-\pi/2, \pi/2]$, s and c represent the sine and cosine functions, respectively. $\vec{p}_o^W$ denotes the position vector of the object. Assuming the position coordinates of the object relative to W are x , y , and z , the position vector can be expressed as:

$$\vec{p}_o^W = [x \quad y \quad z]^T \quad (15)$$

The continuity of R_o^W is ensured by the continuous nature of the sine and cosine functions. Furthermore, when $\beta \in (-\pi/2, \pi/2]$, the element $r_{13} = \sin \beta$ remains strictly monotonic, ensuring that R_o^W is both continuous and unique. Additionally, given $\vec{r}_1$ and $\vec{r}_2$, $\vec{r}_3$ can be calculated through the cross product:

$$\vec{r}_3 = \vec{r}_1 \times \vec{r}_2 = \begin{bmatrix} r_{11} \\ r_{21} \\ r_{31} \end{bmatrix} \times \begin{bmatrix} r_{12} \\ r_{22} \\ r_{32} \end{bmatrix} \quad (16)$$

Since $\vec{p}_o^W$ is a 3D vector in the Cartesian coordinate system, it guarantees both uniqueness and continuity, allowing us to represent the pose as a 9D tuple: $CPR = (x, y, z, r_{11}, r_{21}, r_{31}, r_{12}, r_{22}, r_{32})$. This representation is continuous and unique. Additionally, as shown in Eq. (13) to (16), the CPR can be directly converted into the homogeneous transformation matrix, from which the target pose can be reconstructed.

Building on the above, the agent observes a state comprising the pose of the end-effector $CPR_{ee} \in \mathbb{R}^9$, the pose of target point $CPR_t \in \mathbb{R}^9$, and the pose error $CPR_{err} \in \mathbb{R}^9$. To accelerate the algorithm's convergence, the position error d_{pos} and orientation error d_{ori} are incorporated into the state space, providing intuitive information about pose discrepancies:

$$d_{pos} = \|\vec{p}_{ee}^W - \vec{p}_t^W\|_2 \quad (17)$$

$$d_{\text{ori}} = \arccos \left(\frac{\text{tr} \left(R_{\text{ee}}^W (R_t^W)^T \right)}{2} \right) \quad (18)$$

where $\|\cdot\|_2$ represents the 2-Norm of the vector, and $\text{tr}(\cdot)$ represents the trace of a square matrix. Additionally, the joint angles $\Theta \in \mathbb{R}^6$ and angular velocities $\dot{\Theta} \in \mathbb{R}^6$ of the space manipulator are also included into the state space to ensure the continuity and stability of the manipulator's trajectory planning process.

The action space is also designed to align with the task requirements. In this study, the action is defined as $a = [\dot{\Theta}] \in \mathbb{R}^6$. Joint velocities are chosen as the action space to effectively avoid discontinuities in joint angles.

4.1.3. Reward function

The reward function aims to guide the agent in finding a trajectory that enables the end-effector to reach the target pose with both stability and efficiency. To achieve this, we have developed a dense reward function that encourages both accurate end-effector pose attainment and smooth joint angular velocities. The dense reward in this study comprises the following four components:

- 1 **Pose error penalty:** This penalty is based on the pose error between the end-effector and the target. It is calculated as a weighted sum of the position and orientation errors:

$$p_{\text{pose}} = k_{\text{pos}}^p \cdot d_{\text{pos}} + k_{\text{ori}}^p \cdot d_{\text{ori}} \quad (19)$$

where k_{pos}^p and k_{ori}^p represent the weight factor of the position and orientation error, respectively. In our work, we choose $k_{\text{pos}}^p = 0.5$ and $k_{\text{ori}}^p = 0.125$.

- 2 **Smoothness penalty:** This penalty is applied based on changes in joint angular velocities of the manipulator, discouraging abrupt variations in angular velocities:

$$p_{\text{smooth}} = k_{\text{smooth}}^p \cdot \sum_{i=1}^6 d_{\dot{\theta}_i} \quad (20)$$

where $k_{\text{smooth}}^p = 0.1$ is the coefficient for the smoothness penalty, $d_{\dot{\theta}_i}$ is related to the variation in joint angular velocity:

$$d_{\dot{\theta}_i} = \begin{cases} |\dot{\theta}_i^t - \dot{\theta}_i^{t-1}| - 3, & \text{if } |\dot{\theta}_i^t - \dot{\theta}_i^{t-1}| > 3 \\ 0, & \text{otherwise} \end{cases} \quad (21)$$

- 3 **Orientation proximity reward:** This reward is given based on the differential term of the orientation error between the end-effector and the target, encouraging the end-effector to move toward the correct orientation:

$$r_{\text{ori}} = k_{\text{ori}}^r \cdot (d_{\text{ori}}^{t-1} - d_{\text{ori}}^t) \quad (22)$$

where $k_{\text{ori}}^r = 0.15$ is the coefficient for the orientation proximity reward.

- 4 **Completion reward:** This reward is granted to the agent when the position or orientation of the end-effector falls within the specified error threshold, encouraging the agent to achieve and maintain the desired target pose:

$$r_{\text{done}} = k_{\text{done}}^r \cdot (\eta_{\text{pos}} + \eta_{\text{ori}}) \quad (23)$$

where $k_{\text{done}}^r = 0.1$ is the coefficient for the completion reward, η_{pos} and η_{ori} are parameters related to the position and orientation errors, which depend on the current error and the threshold:

$$\eta_{\text{pos}} = \begin{cases} \frac{\epsilon_{\text{pos}} - d_{\text{pos}}}{\epsilon_{\text{pos}}}, & \text{if } d_{\text{pos}} < \epsilon_{\text{pos}} \\ 0, & \text{otherwise} \end{cases} \quad (24)$$

where ϵ_{pos} is the threshold of the position error. When the position error falls below this threshold, the end-effector position is deemed to have reached the target position. A similar approach is applied to the orientation error. Further detailed regarding ϵ_{pos} and ϵ_{ori} are elaborated in Section 4.2.2 below.

Finally, the dense reward function is formulated as follows:

$$R_{\text{dense}} = -p_{\text{pose}} - p_{\text{smooth}} + r_{\text{ori}} + r_{\text{done}} \quad (25)$$

4.2. Hybrid efficient learning framework

For on-policy DRL algorithms, the distribution of successful and failed trajectories is often highly imbalanced, with successful trajectories being relatively rare. This imbalance is amplified in high-dimensional space manipulation systems. The scarcity of successful trajectories, combined with changing target poses during the whole training, severely constrains the performance of on-policy DRL algorithms. To overcome this issue, we propose a Hybrid Efficient Learning framework (HybridEL), which leverages Hindsight Experience Replay and Curriculum Learning to balances the distribution of successful and failed trajectories, thereby improving the algorithm's convergence.

4.2.1. HER for on-policy learning algorithm

To increase the number of positive samples for the learning algorithm, we utilized the HER method [38] in the early stages of training to enhance sample efficiency. Specifically, this approach assumes that the pose of end-effector $\mathcal{P}_{\text{ee}}^m$ reached at episode m serves as the target pose for the subsequent episode. That means ($\mathcal{P}_{\text{ee}}^m \Rightarrow \mathcal{P}_i^{m+1}$). In episode $m+1$, the action sequence is maintained as episode m , and the reward function is recalculated based on the new state-action pairs, thereby generating successful experiences.

As mentioned earlier, HER generates state-action pairs for episode $m+1$ based on the policy from episode m . However, on-policy algorithms can only learn from transitions collected under the same policy. This inherent "one-episode gap" between past experiences and model parameters limits the effectiveness of HER for on-policy algorithms. To address this limitation, we drew inspiration from OpenAI's approach [42] and built an experience buffer to store transitions generated under the same model parameters. The learning process is divided into two stages: in the first stage, samples are collected using the same policy to populate the experience buffer; in the second stage, transitions are sampled from the buffer to update the model. This two-stage approach effectively approximates the first stage as an off-policy setup, thereby making HER applicable to on-policy algorithms.

It is important to note that while HER facilitates the stable collection of successful trajectories, the optimality of these trajectories is not guaranteed. Moreover, relying on HER can partially restrict the agent's exploration capabilities. As a result, we use HER only during the early stages of training. Once the training progresses reach a certain point, we fully disable HER to ensure that the agent's exploration capabilities are no longer constrained in the later stages.

4.2.2. Precision-based curriculum learning

Curriculum learning (CL) [39] is a technique that enhances training efficiency by enabling the agent to gradually acquire problem-solving skills through a sequence of increasingly challenging tasks. The success of CL largely depends on the design of the curriculum. In this study, we propose a precision-based CL algorithm. Specifically, we allow a larger initial pose error ϵ_{pose}^0 during the early stages of training, making it easier for the agent to complete tasks and receive rewards. As the agent's capabilities improving, we gradually reduce the allowable error range, increasing the precision requirements until reaching the final target error ϵ_{pose}^* . In this study, the pose error threshold decays exponentially as follows:

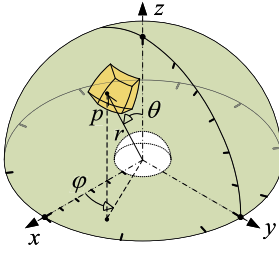


Fig. 3. Partitioning the flexible workspace of a space robotic manipulator based on spherical polar coordinates. The green area represents the flexible workspace $\mathcal{W}$, the yellow region denotes a subspace divided based on spherical polar coordinates, and the point p marks the position of the target within the subspace. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

$$\epsilon_{\text{pose}}^{m+1} = \max \left(\epsilon_{\text{pose}}^m \odot \gamma_{\text{pose}}, \epsilon_{\text{pose}}^* \right) \quad (26)$$

where ϵ_{pose} and γ_{pose} represent the pose error (including both position and orientation) and the precision decay factor, respectively. Additionally, ϵ_{pose}^* denotes the lower bound of the pose error. The symbol $\odot$ represents the Hadamard product. In this study, we set the hyperparameters of the CL as $\epsilon_{\text{pose}}^0 = [0.1, 0.3]$, $\gamma_{\text{pose}} = [0.8, 0.8]$ and $\epsilon_{\text{pose}}^* = [0.05, 0.1]$ meaning that the learning process can be divided into five distinct precision levels.

The success rate is used as a metric to advance the curriculum. First, the space manipulator's flexible workspace, denoted as $\mathcal{W}^{m \times n \times p}$, is divided into multiple subspaces $w_{i,j,k}$, where $i \in [0, 1, \dots, m-1]$, $j \in [0, 1, \dots, n-1]$, and $k \in [0, 1, \dots, p-1]$. In each episode, a subspace is randomly selected, and a target pose is generated for training. If the agent achieves the target pose within a specified precision, the success rate for that subspace is incremented. When the success rate across all subspaces reaches a predefined threshold, the current curriculum stage is considered complete. The success rates are then reset, and the pose accuracy requirement is adjusted to initiate the next stage of curriculum training. This process is repeated until the overall training is complete. In this study, the flexible workspace $\mathcal{W}$ is divided into $5 \times 6 \times 10$ subspaces w based on spherical polar coordinates, with r , θ , and ϕ segmented into 5, 6, and 10 parts, respectively, as shown as Fig. 3. We also tested alternative subdivision settings to examine their impact, and the results suggest that our proposed method is relatively robust to moderate changes in the workspace segmentation. A success rate threshold of 0.85 is set for curriculum progression, which empirically balances task performance and the relative duration of each curriculum stage.

4.3. RTPC architecture

As discussed above, the RTPC algorithm with HybridEL framework is shown as Fig. 4. The RTPC algorithm is implemented with a 3-layer fully connected neural network, where the input vector has 41 dimensions. The hidden layers have sizes of 256, 256, and 128 neurons, and the actor and critic networks produce outputs of 6 and 1 dimensions, respectively. Each hidden layer is followed by a tanh activation function. For the actor network, the 6-dimensional output is bounded using the tanh function to lie within the range of $(-1, 1)$. Given that the manipulator's joint angular velocity limits are set to ± 3 rad/s, the output of the actor network is scaled by a factor of 3 to yield the input angular velocity for the manipulator.

During the training process, various states of the space manipulator are collected from the simulation environment, including the poses of the end effector and target, as well as the joint angles and angular velocities. The orientations of these poses are represented using quaternions, which, along with the Cartesian position vectors, are used to calculate the continuous pose representation. A similar approach is applied to compute the continuous pose representation of the pose error. The input tensor for RTPC undergoes state normalization preprocess-

Table 2

Parameters of free-floating space manipulator system.

Component	Parameters	Values
Base Spacecraft	Mass [kg]	100
	Offset vector [m]	(0, 0, 0)
	Size [m]	(1.2, 1.2, 1.2)
	Inertia matrix [kg · m ²]	$\text{diag}(41.6, 52.9, 52.9)$
Manipulator	Offset vector [m]	(0, -0.4, 0.6)
	Joint angle range [rad]	$(\pm 2\pi, \pm 2\pi, \pm 2\pi, \pm \pi, \pm \pi, \pm \pi)$
	Joint angular velocity range [rad/s]	$(\pm 3, \pm 3, \pm 3, \pm 3, \pm 3, \pm 3)$

Table 3

Hyperparameters of learning-based algorithms.

Hyperparameters	RTPC / Vanilla PPO	DDPG / TD3	DNN
Actor network	(256, 256, 128)	(256, 256, 128)	(256, 256, 128)
Critic network	(256, 256, 128)	(256, 256, 128)	(256, 256, 128)
Activation function	Tanh	Tanh	Tanh
Learning rate of actor	2e-4	2e-4	–
Learning rate of critic	1e-4	1e-4	–
Learning rate	–	–	2e-4
Optimizer	Adam	Adam	Adam
Experience buffer size	6e4	–	–
Sample batch size	6e3	–	–
Replay buffer size	–	1e6	–
Batch size	–	64	3200
Training episodes (M)	3.6e5	3.6e5	1e5
Steps per episode (N)	50	50	50
Discount factor (γ)	0.94	0.94	–
TD decay factor (λ)	0.95	–	–
PPO clipping factor (ϵ)	0.2	–	–
HER epochs	100 / –	–	–

ing, where the mean and standard deviation are dynamically computed during training to ensure that the input states follow a standard normal distribution. Additionally, reward scaling is applied by normalizing the rewards with the standard deviation of a rolling discounted sum [43]. The experience buffer is set to a size of 60000, which is filled after 1200 trajectory planning episodes. The advantage function is then computed using Generalized Advantage Estimation (GAE), and 6000 transitions are randomly sampled for training. We employ the Adam optimizer for gradient descent and apply clipping to ensure stable policy updates for the actor network. This sampling process is repeated 45 times, after which the experience buffer is cleared. Once the agent update is complete, the success rate of the most recent 1200 trajectory planning episodes is calculated. If it meets the predefined threshold, the current curriculum is considered complete, and the next curriculum begins, which involves lowering the pose error threshold. This cycle continues until all training episodes are finished.

5. Experiments

5.1. Environment settings

We constructed a simulation environment for the free-floating space manipulator using the PyBullet physics simulation platform, as shown in Fig. 5. The UR5 robotic arm is selected to serve as a 6-DoF space manipulator, which is rigidly attached to the base spacecraft. The target pose space is defined as a hollow hemispherical region centered at the base of the manipulator, with an inner radius of 0.25 m and an outer radius of 0.65 m. Target poses were sampled uniformly in spherical polar coordinates. The kinematic and dynamic parameters of the robotic arm are the official default values. The manipulator operates in joint velocity control mode, with the PyBullet environment configured to simulate a timestep of 0.1 seconds per command. The trajectory planning task is regarded as successful when the position error ϵ_{pos} between the end-effector and the target remains below 0.05 m, and the orientation error

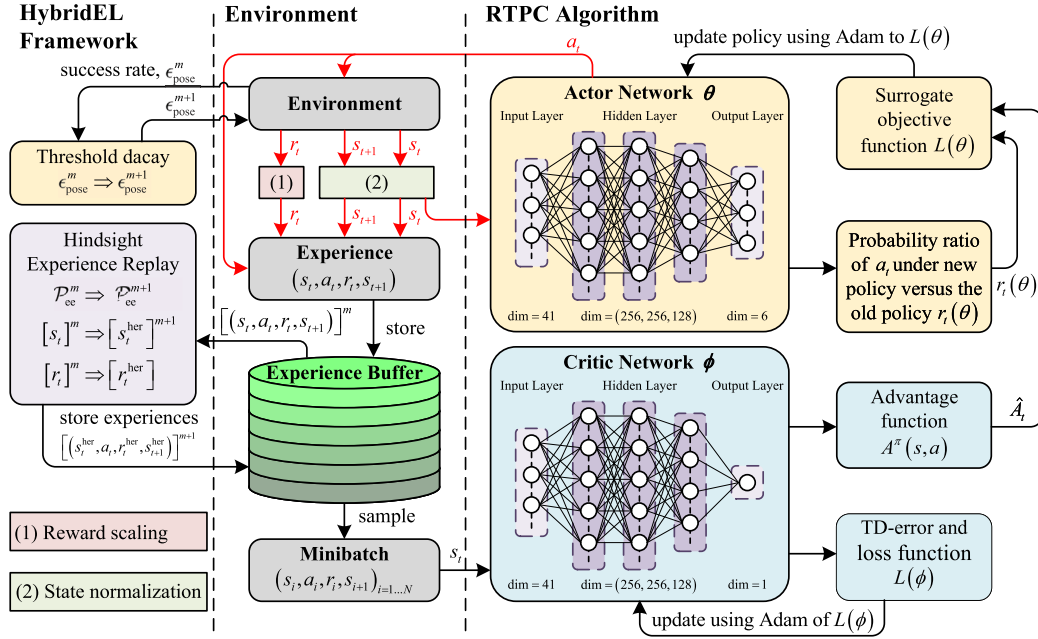


Fig. 4. The RTPC algorithm with HybridEL framework.

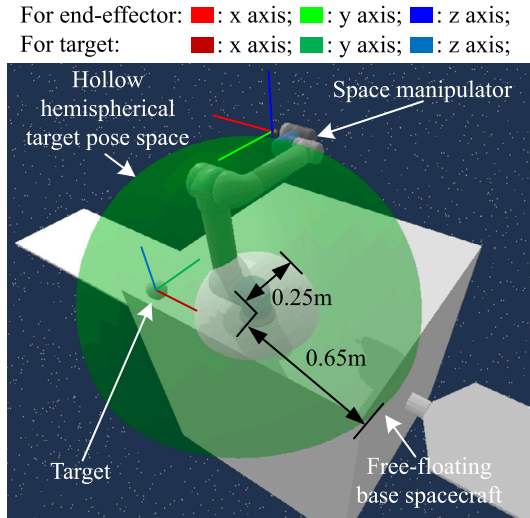


Fig. 5. Simulation environment of the free-floating space manipulator system.

ϵ_{ori} remains below 0.1 rad, continuously for five timesteps. The parameters of the base spacecraft and manipulator are shown in Table 2.

5.2. Performance evaluation

In this part, we first compare our RTPC algorithm with baseline DRL algorithms to demonstrate the advancement of our method. Next, we evaluate the trajectory planning performance in the flexible workspace. In addition, a specific target pose is selected to show the effectiveness of the trajectory planning process.

Three DRL-based algorithms are selected as baselines to compare with our RTPC method. Wu et al. [18] introduced **DDPG**-based algorithm for free-floating space manipulator trajectory planning tasks. DDPG is an off-policy DRL algorithm built upon the actor-critic framework. It outputs deterministic actions through the policy network and stabilizes training via target networks and experience replay. Cao et al. [22] employed **TD3** for free-floating space robot manipulation. TD3 is an enhancement of DDPG, incorporating a double Q-network to improve

both training stability and performance. Additionally, we compared our RTPC with **Vanilla PPO**. In addition to the reinforcement learning-based algorithms, Chai et al. [44] presents a novel framework that first uses optimal trajectory planning to generate a database, and then employs a DNN-based network to learn the mapping from states to controls. However, optimal trajectory planning for a free-floating space manipulator is challenging. To address this issue, we used a fixed-base model to generate the trajectory planning dataset, and evaluated the performance in a free-floating environment. We implemented and trained the baseline algorithms from scratch within our environment. The hyperparameters for both our method and baseline methods are summarized in Table 3. The baseline algorithms use Cartesian position and Quaternion orientation-based Pose Representation (C-QPR) in the state space, while the action space and reward function remain unchanged. We evaluate the trajectory planning process using four metrics: Average Reward (AR), Average Success Rate (ASR), Average Position Error (APE), and Average Orientation Error (AOE).

Fig. 6(a) presents the AR curves for these algorithms throughout the training process. Our method achieves the highest AR values across the entire training duration. As the precision requirements increase, the AR remains positive in the later stages of training. Notable drops in the AR curve occur at epochs 100 and 110, corresponding to the stop of HER and the start of a new curriculum, respectively. In comparison to Vanilla PPO, both TD3 and DDPG exhibit higher rewards during the early stages of training, likely due to their deterministic action outputs. Among these, TD3 outperforms DDPG, which is widely adopted in space manipulator trajectory planning. Vanilla PPO displays the lowest AR among all the algorithms, only approaching DDPG's AR towards the end of the training.

Fig. 6(b) shows the ASR curves for these methods. Due to the selection of a relatively easier curriculum, RTPC algorithm shows a rapid increase in ASR during the early training phases. At epoch 110, a curriculum change leads to a sharp decline in ASR, followed by a gradual recovery in the subsequent epochs. This periodic fluctuation, which occurs five times, corresponds to five curriculum transitions. After the fifth curriculum adjustment, the ASR stabilizes at approximately 0.93, with no further significant decreases. In contrast, due to the initially high precision requirement, TD3 exhibits a slower increase in ASR during the early stages, eventually converging to around 0.68 in the mid-training phase, where it remains for the duration of the training. Despite its

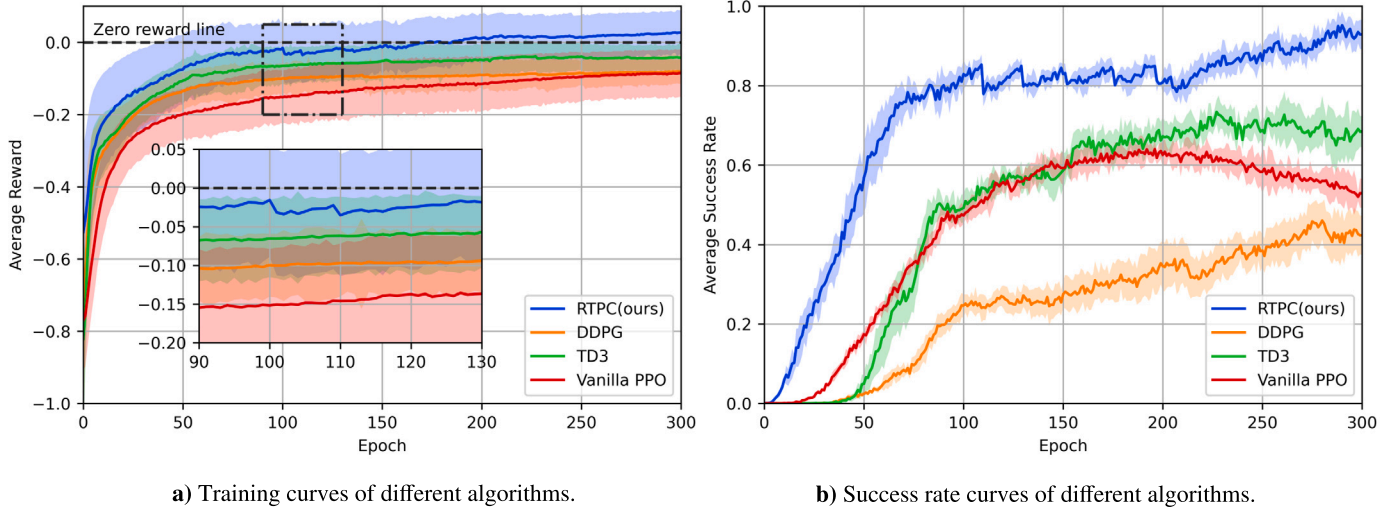


Fig. 6. Performance comparison for different algorithms during training. (Each curve reports the mean value under 5 random seeds, and the shadow area shows the variance.)

widespread use in various studies, DDPG achieved a comparatively poor ASR of 0.42 in the task addressed in this paper.

Additionally, we randomly selected a target pose to illustrate the variation of position and orientation errors during the trajectory planning process, as shown in Fig. 7. The target position is $\text{pos}_t = [0.122, -0.777, 0.763]$ and its orientation (represented as a quaternion) is $\text{ori}_t = [0.128, 0.525, 0.209, 0.815]$. Our RTPC algorithm achieved the best trajectory planning performance, with position and orientation errors stabilizing from step 20 onward. In comparison, Vanilla PPO performed well in position error but consistently exhibited large orientation errors, failing to achieve performance requirements. The DNN algorithm showed significant oscillations starting from step 15, likely due to disturbances caused by changes in the pose of base spacecraft. These experiments demonstrate that our RTPC algorithm outperforms other methods in the free-floating space manipulator task.

After 300 training epochs, we evaluated our RTPC algorithm in comparison with baseline methods by randomly selecting 1000 target poses from the pose space. The evaluation was conducted on a system with an I5-12600KF CPU and an RTX3060 GPU running Ubuntu 18.04. Table 4 summarizes the results, highlighting that RTPC outperforms the baseline methods across most metrics, demonstrating its effectiveness in trajectory planning and fulfilling the real-time requirements. These results were obtained by averaging performance over 5 training using different random seeds. Our data were subjected to one-sided Wilcoxon signed-rank tests, which confirm that the observed differences across algorithms are statistically significant ($p = 0.03125 < 0.05$). RTPC achieves an AR of 0.031 and an ASR of 0.926, along with precise trajectory planning results, featuring an APE of 0.036 and an AOE of 0.065. Notably, the AOE achieved by RTPC is nearly half of the lowest error recorded by other methods, while maintaining the APE within an acceptable threshold, further underscoring its superior performance. Through Cohen's d analysis, we demonstrate that RTPC delivers substantially higher AR and ASR, markedly lower AOE, and maintains competitive APE compared to all baselines. In comparison, TD3 exhibits weaker results than RTPC. While DDPG significantly reduces position error, it shows the poorest convergence in orientation. Similarly, Vanilla PPO struggles with orientation accuracy and, due to the inherent randomness in action selection, ultimately achieves an ASR of only 0.53. Additionally, the DNN algorithm achieves the highest speed at 1092.50 Hz. However, since the dataset was built in a fixed-base environment, the DNN algorithm exhibits the worst APE performance and shows medium AOE and ASR results compared to other algorithms.

Fig. 8 illustrates the trajectory planning process of RTPC for a target pose randomly sampled from the flexible workspace as an example. The

target position is $\text{pos}_t = [0.122, -0.777, 0.763]$ and its orientation (represented as a quaternion) is $\text{ori}_t = [0.128, 0.525, 0.209, 0.815]$. The manipulator reaches the target pose at the 20th step and maintains this accuracy throughout the trajectory process. Meanwhile, the joint angular velocities of space manipulator remain within the rated range of the manipulator. Fig. 8(c) depicts the orientation change of the free-floating base during the trajectory planning process. As the manipulator's position is skewed toward the rear of the spacecraft, the center of gravity shifts backward after task completion, causing a noticeable change in the pitch axis orientation. Nevertheless, the manipulator maintains the required pose accuracy, demonstrating the robustness of the proposed algorithm. Some snapshots of the key steps in one trajectory planning process is provided in Fig. 8(d).

5.3. Additional comparative experiments

5.3.1. Ablation experiments

To investigate the contribution of continuous pose representation (CPR) and HybridEL framework, we conducted ablation studies by removing these parts from RTPC method. The results of these experiments are presented in Fig. 9, while Table 5 shows the performance of the trained models in reaching randomly generated target poses across 1000 trials.

1) Effect of CPR: In the first experiment, we removed CPR and employed C-QPR to assess the performance of the method. As shown in Fig. 9(a) and (b), the impact of CPR on both the AR and ASR is minimal during the early to mid-training stages. However, after the fourth curriculum phase begins, the method using C-QPR fails to exhibit further improvements in either AR or ASR. The AOE of using C-QPR eventually stabilizing at approximately 0.167, roughly 2.5 times higher than the error by using CPR. These findings highlight the significant role of CPR in reducing the orientation error and ensuring the convergence of the model. Specifically, CPR prevents discontinuities and non-uniqueness in the pose representation, thereby enhancing training stability and performance.

2) Effect of HybridEL framework: The HybridEL framework significantly enhances the AR values during the early stages of training. As shown in Fig. 9(a), the AR curve of algorithm that does not employ the HybridEL framework is only slightly better than that of Vanilla PPO, falling short of the performance achieved by RTPC. Due to the integration of CPR, algorithms that omit HybridEL still achieve better ASR and AOE performance throughout the entire process compared to Vanilla PPO, but ultimately perform worse than other methods. This ablation study highlights the essential role of HybridEL in improving model per-

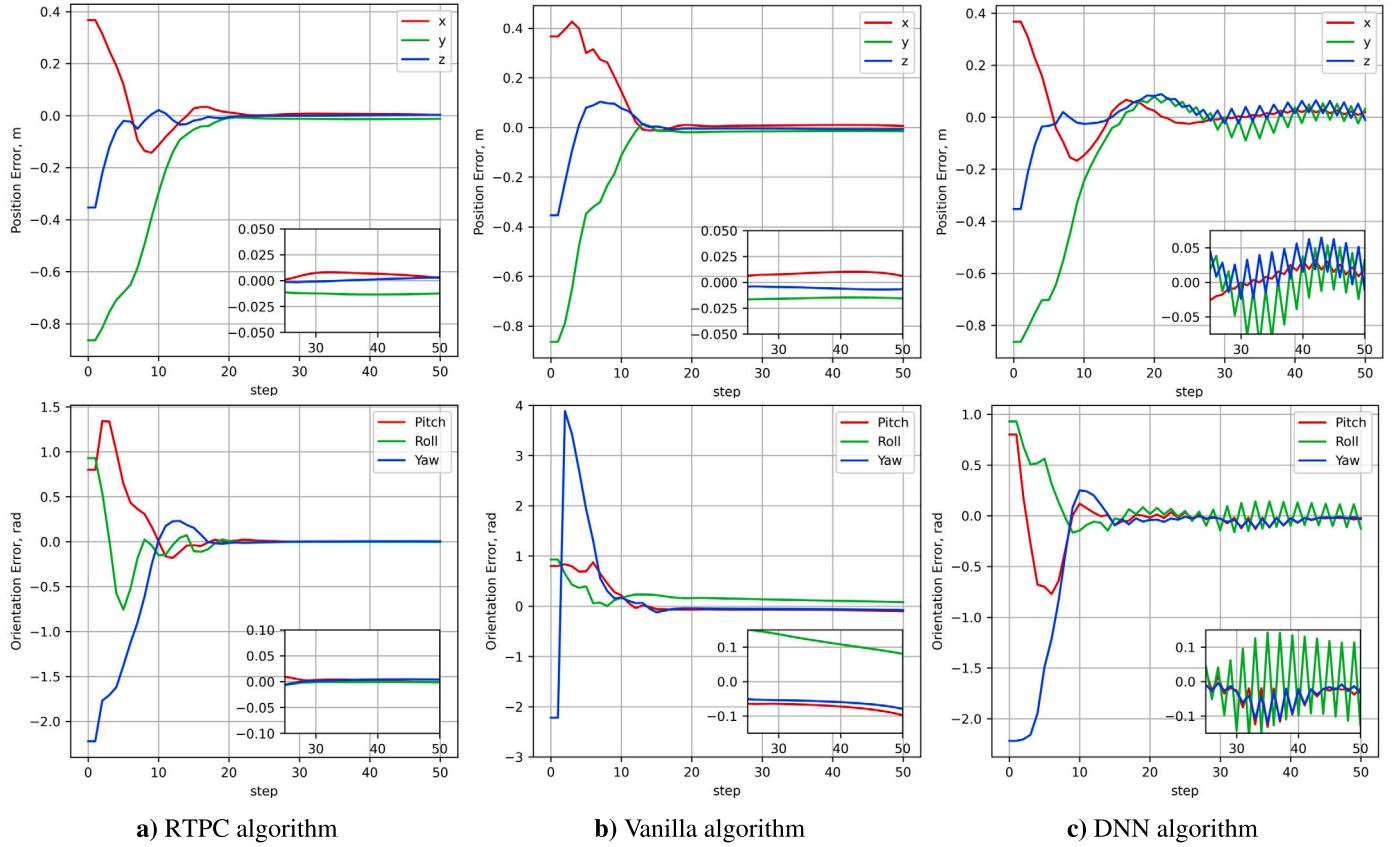


Fig. 7. The position and orientation error of the manipulator in trajectory planning with different algorithms.

Table 4

The performance of different learning algorithms on trajectory planning.

Algorithm	AR	ASR	APE [m]	AOE [rad]	Speed [Hz]
RTPC (ours)	0.031 ± 0.022	0.926 ± 0.018	0.036 ± 0.015	0.065 ± 0.051	724.87
DDPG	-0.079 ± 0.010	0.419 ± 0.021	0.023 ± 0.004	0.252 ± 0.029	998.78
TD3	-0.045 ± 0.014	0.682 ± 0.019	0.034 ± 0.005	0.128 ± 0.018	1056.26
Vanilla PPO	-0.082 ± 0.023	0.527 ± 0.018	0.026 ± 0.012	0.244 ± 0.059	750.74
DNN	–	0.605 ± 0.023	0.045 ± 0.014	0.146 ± 0.035	1092.50

Table 5

Effect of key components in RTPC on trajectory planning performance.

Algorithm	AR	ASR	APE [m]	AOE [rad]
Original	0.031 ± 0.022	0.926 ± 0.018	0.036 ± 0.015	0.065 ± 0.051
W/o CPR	-0.035 ± 0.019	0.757 ± 0.020	0.034 ± 0.014	0.167 ± 0.063
W/o HybridEL	-0.053 ± 0.021	0.634 ± 0.029	0.025 ± 0.012	0.212 ± 0.042
Vanilla PPO	-0.082 ± 0.023	0.527 ± 0.018	0.026 ± 0.012	0.244 ± 0.059

formance, as it facilitates the acquisition of sufficient positive samples early in training, thereby accelerating task completion.

5.3.2. Effect of CPR and HER on baseline methods

To isolate the effects of CPR and HER, we further evaluated DDPG and TD3 baselines with both components integrated but without curriculum learning. These variants, denoted as **DDPG-CH** and **TD3-CH**, allow for a clearer analysis of individual module contributions. Fig. 10 presents the experimental results, and Table 6 summarizes the models' performance in reaching random target poses over 1000 trials.

Throughout the training process, the variants using CPR and HER consistently outperformed their original counterparts in terms of both AR and ASR. These variants exhibited significantly faster convergence

Table 6

The performance of DDPG/TD3 and their CPR-based HER variants on trajectory planning.

Algorithm	AR	ASR	APE [m]	AOE [rad]
DDPG	-0.079 ± 0.010	0.419 ± 0.021	0.023 ± 0.004	0.252 ± 0.029
DDPG-CH	-0.073 ± 0.010	0.603 ± 0.015	0.022 ± 0.003	0.229 ± 0.023
TD3	-0.045 ± 0.014	0.682 ± 0.019	0.034 ± 0.005	0.128 ± 0.018
TD3-CH	-0.034 ± 0.012	0.773 ± 0.028	0.032 ± 0.006	0.119 ± 0.015

during the first 100 epochs, with TD3-CH showing particularly notable improvement. After the HER phase concluded, both DDPG-CH and TD3-CH experienced a minor drop in AR, followed by a gradual recovery that continued until the end of training.

After 300 training epochs, we assessed both DDPG and TD3, along with their CPR-based HER variants, on 1000 randomly sampled target poses from the pose space. The results show that both DDPG-CH and TD3-CH achieved lower APE and AOE compared to their original counterparts, indicating that the incorporation of CPR and HER contributes to more accurate trajectory planning in terms of both position and orientation. These findings highlight the effectiveness of both CPR and HER in improving trajectory planning performance. Specifically, HER

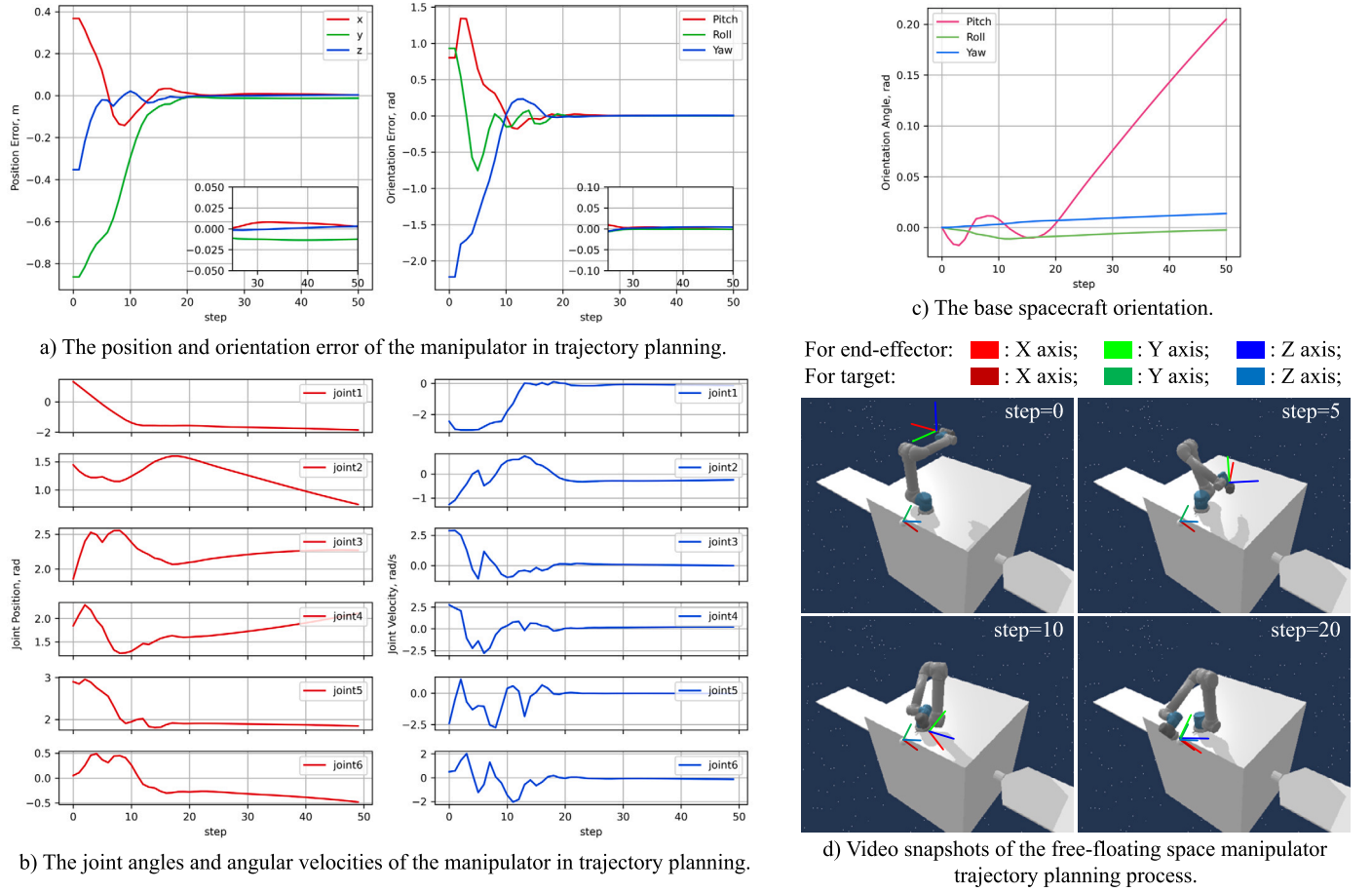


Fig. 8. The trajectory planning process of free-floating space manipulator using RTPC algorithm.

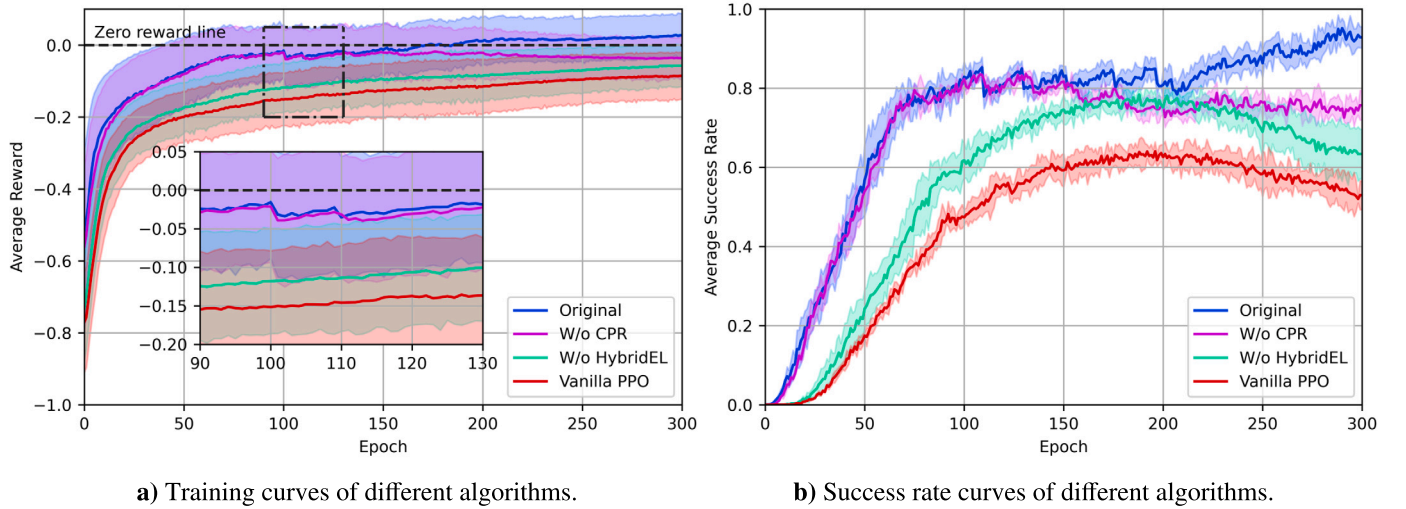


Fig. 9. Performance comparison of ablation study variants during training.

notably accelerates early-stage convergence, while CPR substantially reduces orientation errors. Importantly, the benefits of CPR and HER are not limited to our proposed RTPC algorithm but also extend to other DRL-based approaches, demonstrating strong potential for broader applicability.

5.3.3. Sparse reward function evaluation

The design of dense reward functions is complex and relies on precise observational information, which may not be feasible in certain

scenarios. To address this, we followed the approach outlined in [45] and designed a sparse reward function based solely on pose error, as defined by:

$$R_{\text{sparse}} = \begin{cases} 1, & \text{if } d_{\text{pos}} < \epsilon_{\text{pos}}, d_{\text{ori}} < \epsilon_{\text{ori}}, \\ 0, & \text{otherwise} \end{cases} \quad (27)$$

where d_{pos} , d_{ori} , ϵ_{pos} and ϵ_{ori} are consistent with the definitions provided in Section 4.1.3. Fig. 11 compares the average pose error curves dur-

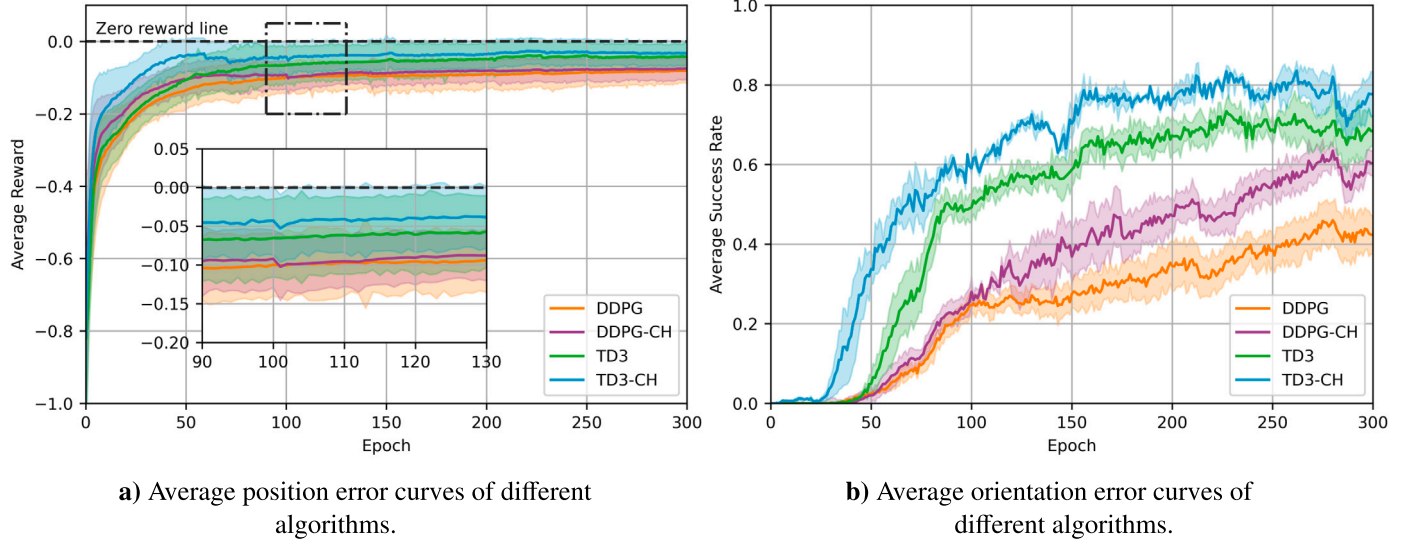


Fig. 10. Performance comparison of DDPG/TD3 and their CPR-based HER variants.

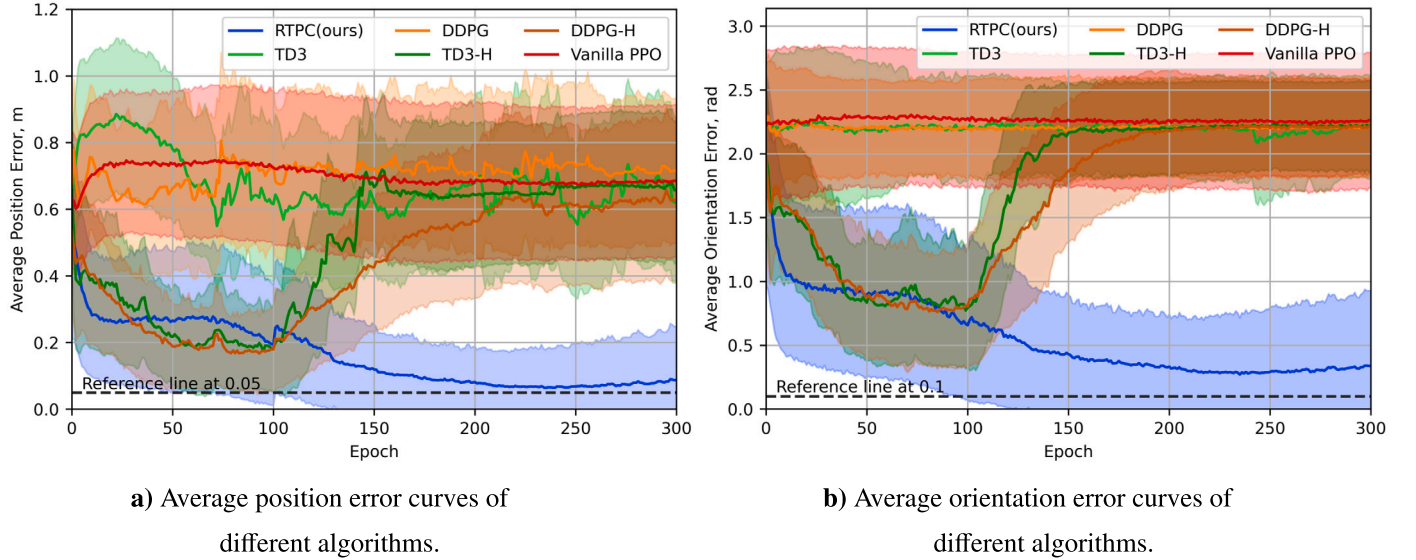


Fig. 11. Performance comparison of different algorithms with a sparse reward function.

Table 7

The performance of different learning algorithms with a sparse reward function.

Algorithm	AR	ASR	APE [m]	AOE [rad]
RTPC (ours)	0.045 ± 0.027	0.217 ± 0.013	0.087 ± 0.054	0.338 ± 0.197
DDPG	0 ± 0	0 ± 0	0.715 ± 0.073	2.211 ± 0.129
DDPG-H	0 ± 0	0 ± 0	0.626 ± 0.080	2.213 ± 0.123
TD3	0 ± 0	0 ± 0	0.620 ± 0.080	2.209 ± 0.136
TD3-H	0 ± 0	0 ± 0	0.656 ± 0.073	2.215 ± 0.119
Vanilla PPO	0 ± 0	0 ± 0	0.685 ± 0.076	2.262 ± 0.177

ing training for RTPC algorithm and baseline methods. Table 7 presents the evaluation results of the trained models in achieving randomly generated target poses over 1000 trials. Despite the error consistently remaining above the threshold, our method still achieved the highest pose accuracy throughout the training process with the aid of sufficient positive trajectories.

To further analyze the effect of HER in sparse reward scenarios, we additionally evaluated DDPG and TD3 with HER. For ease of reference, we denote these two variants as **DDPG-H** and **TD3-H** in the

remainder of this paper. In the first 100 epochs, both variants showed significantly improved convergence compared to their original versions. Specifically, position and orientation errors were substantially reduced, and their performance approached that of our proposed RTPC by epoch 100. However, after the HER phase ended, both baselines experienced noticeable performance degradation. This degradation is likely caused by successful experiences in the replay buffer being gradually overwritten by zero-reward transitions. As a result, the agent has reduced access to effective learning signals and again struggles with reward sparsity.

Interestingly, TD3-H showed a faster degradation in performance compared to DDPG-H. We hypothesize that TD3's twin-critic minimum selection, though effective in reducing overestimation, limits learning when positive feedback is sparse. Moreover, orientation performance degrades more severely than position performance. This is consistent with our prior observations in Section 5.2, where baseline methods failed to effectively converge in orientation tasks even with dense rewards. By the end of training, both HER-enhanced baselines deteriorate to match the performance of their original versions without HER. These findings demonstrate that HER greatly improves early-stage convergence of

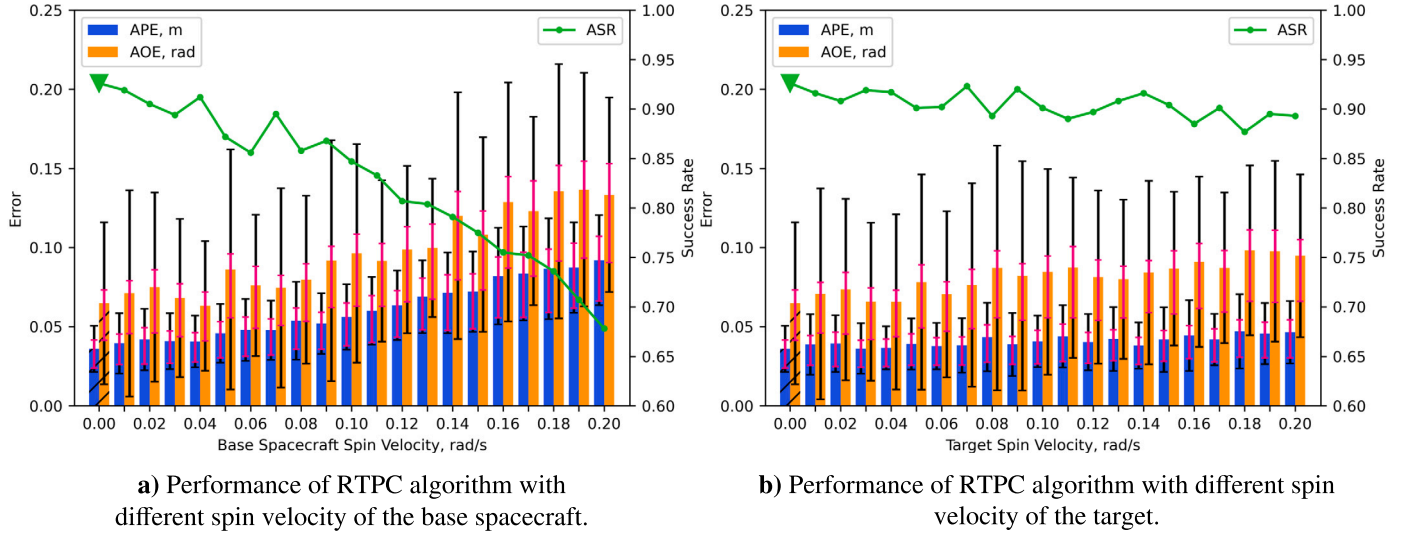


Fig. 12. The performance of RTPC algorithm with different spin velocities of the base spacecraft and the target. (The shadow columns represent the original work state, and the triangle denotes the original ASR. Black error bars represent the standard deviation across 5 random seeds, and deep pink error bars represent the 95% confidence interval.)

DRL algorithms, especially in sparse reward scenarios. However, maintaining its effectiveness over long training may require complementary strategies, such as the curriculum learning approach introduced in our HybridEL framework.

5.4. Robustness evaluation

Robustness and adaptability are essential for the successful execution of space missions. In the space environment, space manipulators are subject to a wide range of disturbances, including but not limited to base spacecraft or target spin, sensor noise, and the delay of control commands or observation data. To provide a comprehensive assessment of the RTPC algorithm's robustness, this study investigates its performance under six representative disturbances [46], capturing the key challenges encountered in space trajectory planning. In each experiment, only one disturbance was modified at a time, while all other conditions remained nominal.

5.4.1. Spin of the base spacecraft or target

Ensuring that the base spacecraft or target remains stationary is nearly impossible in space missions. Therefore, we conducted experiments to evaluate the performance of the RTPC algorithm under base spacecraft and target spin conditions.

To examine the impact of base spacecraft spin, we introduced a yaw-axis spin velocity to ensure the target remained within the flexible workspace of the space manipulator despite the base's rotation. The test results, shown in Fig. 12(a), indicate that as the spin velocity increases, both APE and AOE exhibit a significant rise, ultimately leading to a decline in the ASR.

Similarly, we investigated the effect of target spin while keeping the base spacecraft stationary. In this scenario, we introduced an arbitrary-axis spin velocity to the target. The experimental results, presented in Fig. 12(b), demonstrate that the RTPC algorithm exhibits strong robustness against potential target spin. As the spin velocity increases, the required adjustments become more substantial, which leads to a greater AOE. In contrast, APE and ASR exhibit relative stability, as position tracking is less sensitive to rotational disturbances.

5.4.2. Mass variation of the base spacecraft

To evaluate the RTPC algorithm's adaptability to dynamic coupling, we conducted a series of experiments with different base spacecraft

mass, as shown in Fig. 13(a). When the spacecraft mass gradually increased to 200% of its original value, the trajectory planning performance remained highly robust, with no significant increase in APE or AOE. However, as the spacecraft mass decreased, both position and orientation accuracy exhibits a slight decline. Notably, when the mass was halved, the ASR dropped to approximately 0.90. These findings highlight the strong adaptability of our algorithm to varying levels of dynamic coupling, ensuring stable trajectory planning performance even under substantial mass variations.

5.4.3. Disturbance in joint angular velocity

The space manipulator may be subjected to external torques from factors such as spacecraft propulsion jets, resulting in perturbations in joint angular velocity. To evaluate the effect of joint angular velocity disturbances, we applied randomly perturbations to each manipulator joint. The experimental results, shown in Fig. 13(b), indicate that when the angular velocity perturbation remains below 0.2 rad/s, trajectory planning performance experiences only a minor degradation. However, as the disturbance intensity increases, AOE rises significantly, while ASR declines sharply. When the disturbance reaches 10% of the maximum allowable velocity, the ASR drops below 0.7. These results demonstrate that the RTPC algorithm exhibits robustness against joint angular velocity disturbances within a certain range. However, effectively mitigating larger-scale disturbances remains a critical challenge that needs to be addressed in future work.

5.4.4. Observation noise of the target

Accurate observations are often challenging to obtain in space missions. To examine the impact of observation noise, we conducted a series of experiments, as illustrated in Fig. 14. At the beginning of the trajectory planning process, the observed target position and orientation had significant errors. These errors decreased linearly over time and were entirely eliminated by step 30. The experimental results indicate that mild position noise has a minimal impact on trajectory planning performance. However, as the noise level increases, both the APE and AOE rise, ultimately leading to a decline in the ASR. In contrast, the RTPC algorithm demonstrates greater robustness to orientation noise. An increase in orientation noise primarily affects AOE and ASR, while having a negligible effect on APE. We believe that the RTPC algorithm prioritizes reaching the target's vicinity over strictly maintaining the expected pose during trajectory planning.

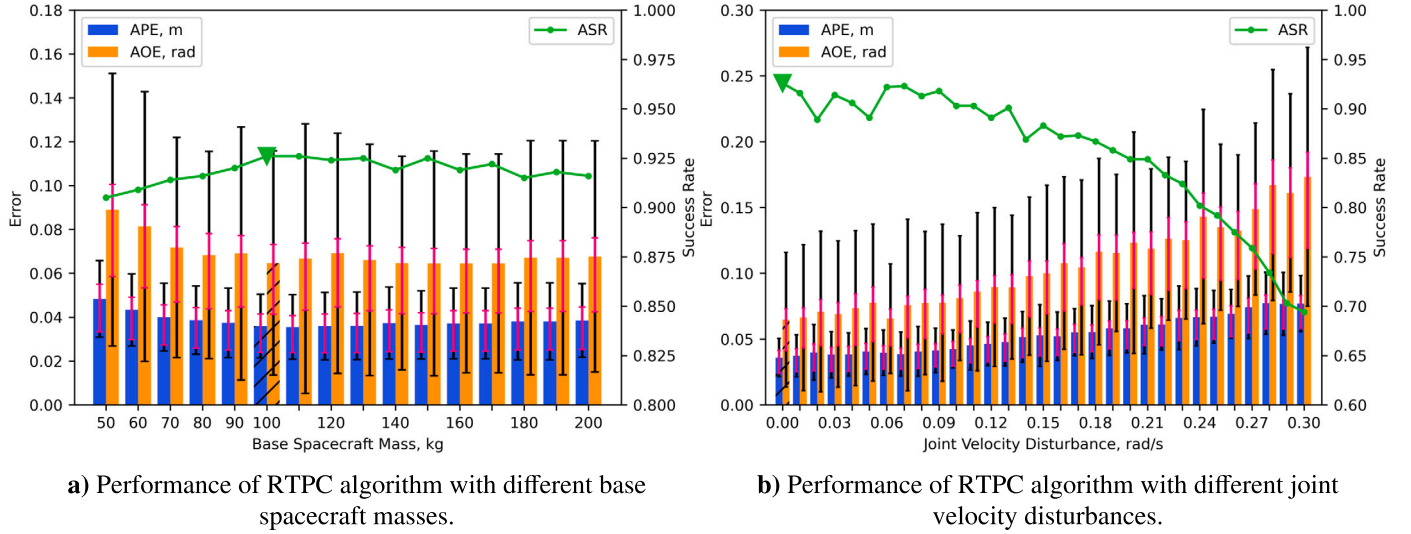


Fig. 13. The performance of RTPC algorithm with different base spacecraft masses and joint velocity disturbances.

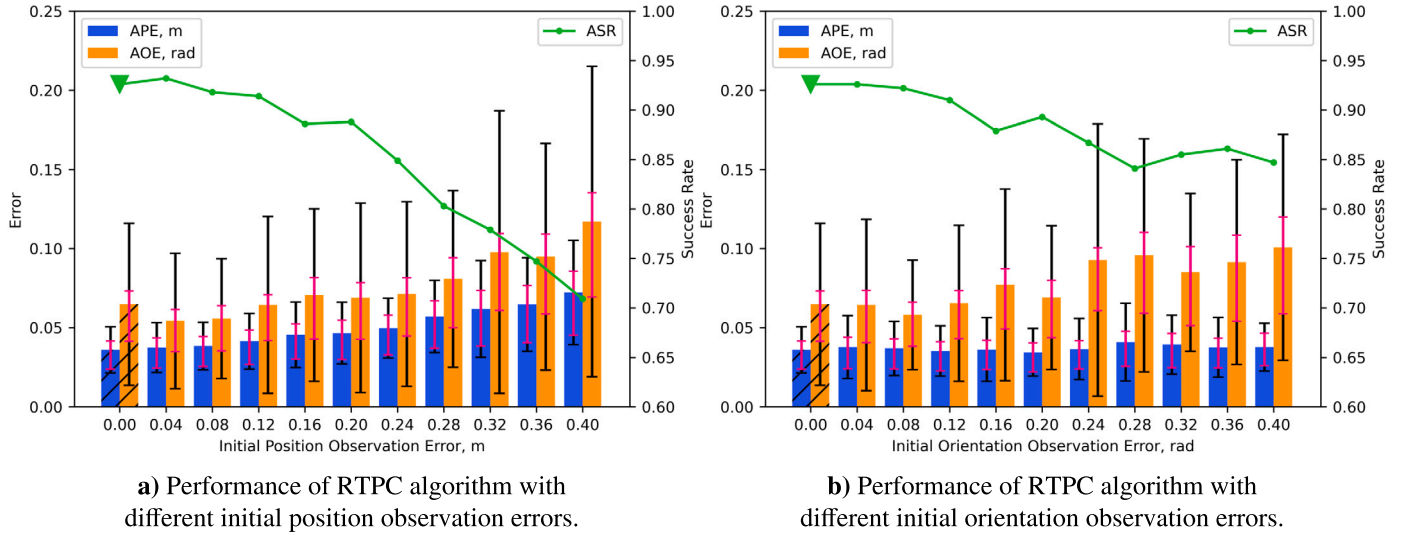


Fig. 14. The performance of RTPC algorithm with different initial position and orientation observation errors.

5.4.5. Control command or observation data delay

The stability of data transmission is crucial for space missions, as the delay of control commands or observation data can severely affect mission success. To evaluate the robustness of our RTPC algorithm, we analyzed the impact of control command and observation data delay on its performance, as illustrated in Fig. 15. Specifically, starting from step 2, we randomly selected certain steps and replaced the current control commands or observation data with those from the previous time step to simulate data delay. The experimental results show that when the control command delay rate remained within 20%, trajectory planning performance was only slightly affected. However, as the delay rate increased, AOE rose significantly, leading to a decline in ASR. When the control command delay rate reached 40%, the ASR dropped to approximately 0.65. A similar trend was observed for observation data delay. These results indicate that the proposed RTPC algorithm maintains stable performance when the control commands or observation data delay rate is within 20%, exhibiting a certain degree of robustness. Nonetheless, as delay rates exceed 20%, the trajectory planning performance declines markedly, highlighting the need for future work to address delay compensation mechanisms.

5.4.6. Variation of gravitational field

Finally, we conducted experiments to investigate the impact of variations in the gravitational field on the RTPC algorithm, as depicted in Fig. 16. As the gravitational field strength increased, trajectory planning performance significantly degraded, reducing both position and orientation accuracy, and ultimately lowering the ASR. We speculate that fluctuations in the gravitational field may have altered the distribution of CPR_{ee} and CPR_t within the state space, thereby diminishing the overall performance of the reinforcement learning algorithm. To mitigate this effect, the development of effective normalization techniques will be a primary focus of our future research efforts.

6. Conclusions and limitations

6.1. Conclusions

In this paper, we proposed the RTPC algorithm to effectively address 6D trajectory planning tasks for free-floating space manipulator. We designed the CPR to eliminate discontinuities and integrated into state space, effectively reducing pose errors in space manipulation tasks. Ad-

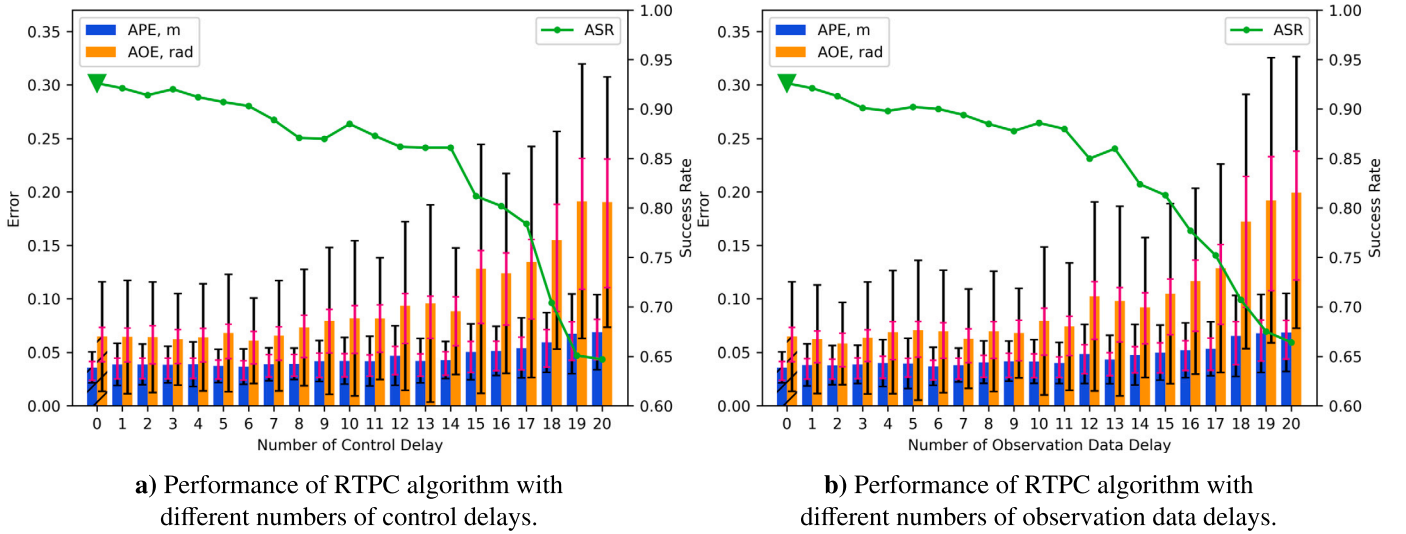


Fig. 15. The performance of RTPC algorithm under control command or observation data delays.

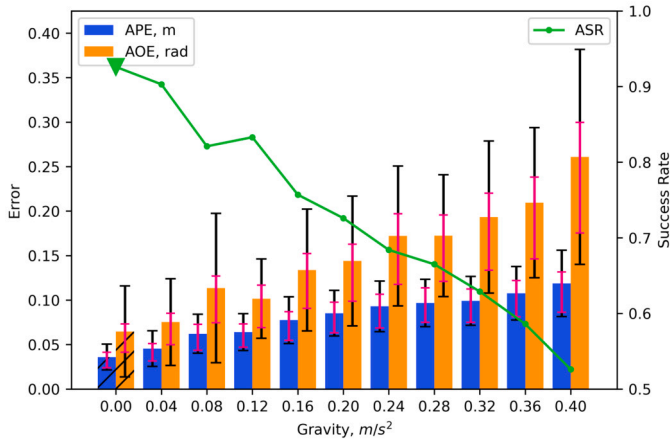


Fig. 16. The performance of RTPC algorithm under different gravitational fields.

ditionally, the HybridEL framework further enhances the agent's ability to acquire efficient experiences during the early and mid-stages, thereby accelerating the convergence of DRL algorithm. A series of comparative experiments demonstrate that the RTPC algorithm significantly enhances both the success rate and accuracy of space manipulator trajectory planning, while also maintaining robust and real-time performance. The evaluation of different types of reward functions highlights the versatility and adaptability of our approach, and extensive disturbance tests demonstrate the robustness of our method. We are pleased to note that the two key innovations, CPR and HybridEL framework, can be independently incorporated into existing reinforcement learning algorithms to enhance their performance. Moreover, HybridEL directly addresses the DRL sample efficiency bottleneck and is readily transferable to other application domains.

6.2. Limitations and future directions

Despite these positive results, we also acknowledge that there remains a significant gap between simulation and real-world deployment for reinforcement learning-based trajectory planning algorithms for space manipulators. In contrast to simulations, real-world environments introduce more complex and time-varying disturbances, and often lack dense, accurate data, which pose substantial challenges for model-

free algorithms. As such, incorporating various disturbances during the training phase is crucial for developing a more robust trajectory planning algorithm and facilitating the transition from simulation to real-world applications.

Furthermore, due to the strict payload limitations of space missions, deploying neural networks on edge devices, such as FPGAs, could offer significant advantages for the application of learning-based algorithms in space environments. In this context, ensuring compatibility with spacecraft safety systems is also a pressing research need. Additionally, space missions often require more redundant sensor designs, such as independent sensors for joint angles, angular velocities, and forces, separate from those integrated into the manipulator. Enhancing the algorithm's performance while integrating such redundant information is a key challenge that warrants further investigation.

In addition, ground-based simulation platforms are essential for algorithm deployment and validation. However, for 6-DoF free-floating space robots, the floating base may experience various reactive forces, including 3-DoF linear and angular velocities, which could render traditional simulation platforms, such as air-bearing tables, unsuitable. To address this, we plan to design a space simulation setup utilizing suspension and cable systems to validate the performance of our algorithm as accurately as possible [47]. This setup further enable more comprehensive evaluations, such as multi-step and long-horizon task scenarios, hardware-in-the-loop testing, and full benchmarking on edge devices. Future work should focus on establishing formal stability and safety guarantees, as well as enhancing the interpretability of the trained policy. We also plan to extend our approach to trajectory planning and tracking for targets with pose variations. Beyond these efforts, we anticipate that RTPC could be adapted in future work to manipulators with varying link lengths or actuator configurations, as well as to other free-floating robotic platforms.

CRedit authorship contribution statement

Yuhui Hu: Software, Conceptualization, Methodology, Writing – original draft, Data curation, Formal analysis. **Dong Zhou:** Methodology, Supervision, Writing – review & editing, Conceptualization. **Weiran Yao:** Supervision, Funding acquisition, Writing – review & editing. **Xiangyu Shao:** Supervision, Funding acquisition. **Guanghui Sun:** Funding acquisition, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was kindly supported by the National Natural Science Foundation of China through grant No. 62403162.

Appendix A. Abbreviations

The abbreviations that appear in this paper can be found in Table 8.

Table 8

Definition of abbreviations. (Listed in the order of appearance.).

Abbreviation	Definition
RTPC	Reinforcement learning-based Trajectory Planning with Continuous pose representation
DoF	Degrees of Freedom
HybridEL	Hybrid Efficient Learning
GJM	generalized Jacobian matrix
DRL	Deep Reinforcement Learning
ZRM	Zero Reaction Maneuver
PSO	Particle Swarm Optimization
DE	Differential Evolution
CPSO	Chaotic PSO
DDPG	Deep Deterministic Policy Gradient
PPO	Proximal Policy Optimization
HER	Hindsight Experience Replay
CL	Curriculum Learning
MDP	Markov Decision Process
RL	Reinforcement Learning
CPR	Continuous Pose Representation
TD	Temporal Difference
GAE	Generalized Advantage Estimation
TD3	Twin Delayed Deep Deterministic policy gradient
DNN	Deep Neural Network
C-QPR	Cartesian position and Quaternion orientation-based Pose Representation
AR	Average Reward
ASR	Average Success Rate
APE	Average Position Error
AOE	Average Orientation Error
DDPG-CH	DDPG with CPR-based HER variant
TD3-CH	TD3 with CPR-based HER variant
DDPG-H	DDPG with HER variant
TD3-H	TD3 with HER variant

Data availability

Data will be made available on request.

References

- [1] B. Ma, Z. Jiang, Y. Liu, Z. Xie, Advances in space robots for on-orbit servicing: a comprehensive review, *Adv. Intell. Syst.* 5 (8) (2023) 2200397.
- [2] X. Zhihui, L. Jinguo, W. Chenchen, T. Yuchuang, Review of in-space assembly technologies, *Chin. J. Aeronaut.* 34 (11) (2021) 21–47.
- [3] A. Ellery, Tutorial review on space manipulators for space debris mitigation, *Robotics* 8 (2) (2019) 34.
- [4] S. Dubowsky, E. Papadopoulos, The kinematics, dynamics, and control of free-flying and free-floating space robotic systems, *IEEE Trans. Robot. Autom.* 9 (5) (1993) 531–543.
- [5] O. Zhang, W. Yao, D. Du, C. Wu, J. Liu, L. Wu, Y. Sun, Trajectory optimization and tracking control of free-flying space robots for capturing non-cooperative tumbling objects, *Aerosp. Sci. Technol.* 143 (2023) 108718.
- [6] R. Chai, A. Tsourdos, A. Savvaris, S. Chai, Y. Xia, Solving constrained trajectory planning problems using biased particle swarm optimization, *IEEE Trans. Aerosp. Electron. Syst.* 57 (3) (2021) 1685–1701.
- [7] J. Ratajczak, K. Tchoń, Lagrangian Jacobian motion planning with application to a free-floating space manipulator, in: 2019 12th International Workshop on Robot Motion and Control (RoMoCo), IEEE, 2019, pp. 166–171.
- [8] R. Chai, A. Tsourdos, A. Savvaris, S. Chai, Y. Xia, C.P. Chen, Review of advanced guidance and control algorithms for space/aerospace vehicles, *Prog. Aerosp. Sci.* 122 (2021) 100696.
- [9] M. Wang, J. Luo, U. Walter, Trajectory planning of free-floating space robot using particle swarm optimization (ps), *Acta Astronaut.* 112 (2015) 77–88.
- [10] M. Wang, J. Luo, J. Fang, J. Yuan, Optimal trajectory planning of free-floating space manipulator using differential evolution algorithm, *Adv. Space Res.* 61 (6) (2018) 1525–1536.
- [11] R. Jin, P. Rocco, Y. Geng, Cartesian trajectory planning of space robots using a multi-objective optimization, *Aerosp. Sci. Technol.* 108 (2021) 106360.
- [12] D. Zhou, G. Sun, W. Lei, L. Wu, Space noncooperative object active tracking with deep reinforcement learning, *IEEE Trans. Aerosp. Electron. Syst.* 58 (6) (2022) 4902–4916.
- [13] D. Zhou, G. Sun, Z. Zhang, L. Wu, On deep recurrent reinforcement learning for active visual tracking of space noncooperative objects, *IEEE Robot. Autom. Lett.* 8 (8) (2023) 4418–4425.
- [14] K. Senda, Y. Tani, Autonomous robust skill generation using reinforcement learning with plant variation, *Adv. Mech. Eng.* 6 (2014) 276264.
- [15] X. Hu, X. Huang, T. Hu, Z. Shi, J. Hui, Mrddpg algorithms for path planning of free-floating space robot, in: 2018 IEEE 9th International Conference on Software Engineering and Service Science (ICSESS), IEEE, 2018, pp. 1079–1082.
- [16] D. Du, Q. Zhou, N. Qi, X. Wang, Y. Liu, Learning to control a free-floating space robot using deep reinforcement learning, in: 2019 IEEE International Conference on Unmanned Systems (ICUS), IEEE, 2019, pp. 519–523.
- [17] R. Zeng, M. Liu, J. Zhang, X. Li, Q. Zhou, Y. Jiang, Manipulator control method based on deep reinforcement learning, in: 2020 Chinese Control and Decision Conference (CCDC), IEEE, 2020, pp. 415–420.
- [18] Y.-H. Wu, Z.-C. Yu, C.-Y. Li, M.-J. He, B. Hua, Z.-M. Chen, Reinforcement learning in dual-arm trajectory planning for a free-floating space robot, *Aerosp. Sci. Technol.* 98 (2020) 105657.
- [19] Y. Li, D. Li, W. Zhu, J. Sun, X. Zhang, S. Li, Constrained motion planning of 7-dof space manipulator via deep reinforcement learning combined with artificial potential field, *Aerospace* 9 (3) (2022) 163.
- [20] A. Al Ali, J.-F. Shi, Z.H. Zhu, Path planning of 6-dof free-floating space robotic manipulators using reinforcement learning, *Acta Astronaut.* 224 (2024) 367–378.
- [21] S. Wang, Y. Cao, X. Zheng, T. Zhang, Collision-free trajectory planning for a 6-dof free-floating space robot via hierarchical decoupling optimization, *IEEE Robot. Autom. Lett.* 7 (2) (2022) 4953–4960.
- [22] Y. Cao, S. Wang, X. Zheng, W. Ma, X. Xie, L. Liu, Reinforcement learning with prior policy guidance for motion planning of dual-arm free-floating space robot, *Aerosp. Sci. Technol.* 136 (2023) 108098.
- [23] R. Srivastava, R. Lima, R. Sah, K. Das, Deep reinforcement learning based control of rotation floating space robots for proximity operations in pybullet, in: 2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC), IEEE, 2023, pp. 1224–1229.
- [24] M. D'Ambrosio, L. Capra, A. Brandonisio, S. Silvestrini, M. Lavagna, Redundant space manipulator autonomous guidance for in-orbit servicing via deep reinforcement learning, *Aerospace* 11 (5) (2024) 341.
- [25] K. Yoshida, K. Hashizume, S. Abiko, Zero reaction maneuver: flight validation with ets-vii space robot and extension to kinematically redundant arm, in: Proceedings 2001 ICRA. IEEE International Conference on Robotics and Automation (Cat. No. 01CH37164), vol. 1, IEEE, 2001, pp. 441–446.
- [26] F. Zhang, Y. Fu, L. Hua, H. Chen, S. Wang, B. Guo, Point-to-point planning for free-floating space manipulator with zero-disturbance spacecraft attitude, in: 2012 IEEE International Conference on Information and Automation, IEEE, 2012, pp. 142–147.
- [27] W. Xu, B. Liang, Y. Xu, Practical approaches to handle the singularities of a wrist-partitioned space manipulator, *Acta Astronaut.* 68 (1–2) (2011) 269–300.
- [28] P.J. From, K.Y. Pettersen, J.T. Gravdahl, Singularity-free dynamic equations of spacecraft-manipulator systems, *Acta Astronaut.* 69 (11–12) (2011) 1057–1065.
- [29] D. Silver, A. Huang, C.J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, et al., Mastering the game of go with deep neural networks and tree search, *Nature* 529 (7587) (2016) 484–489.
- [30] V. Mnih, Playing atari with deep reinforcement learning, *arXiv preprint, arXiv:1312.5602*, 2013.
- [31] Y. Zhu, R. Mottaghi, E. Kolve, J.J. Lim, A. Gupta, L. Fei-Fei, A. Farhadi, Target-driven visual navigation in indoor scenes using deep reinforcement learning, in: 2017 IEEE International Conference on Robotics and Automation (ICRA), IEEE, 2017, pp. 3357–3364.
- [32] K. Arulkumaran, M.P. Deisenroth, M. Brundage, A.A. Bharath, Deep reinforcement learning: a brief survey, *IEEE Signal Process. Mag.* 34 (6) (2017) 26–38.
- [33] A. Brandonisio, L. Capra, M. Lavagna, Deep reinforcement learning spacecraft guidance with state uncertainty for autonomous shape reconstruction of uncooperative target, *Adv. Space Res.* 73 (11) (2024) 5741–5755.
- [34] J. Qi, H. Gao, H. Yu, M. Huo, W. Feng, Z. Deng, Integrated attitude and landing control for quadruped robots in asteroid landing mission scenarios using reinforcement learning, *Acta Astronaut.* 204 (2023) 599–610.
- [35] W. Lei, T. Zhao, G. Sun, Image based target capture of free floating space manipulator under unknown dynamics, *Adv. Space Res.* 72 (11) (2023) 4923–4933.

- [36] Z.-B. Xu, F.-L. Cao, Simultaneous lp-approximation order for neural networks, *Neural Netw.* 18 (7) (2005) 914–923.
- [37] N. Hammami, K.K. Nguyen, On-policy vs. off-policy deep reinforcement learning for resource allocation in open radio access network, in: 2022 IEEE Wireless Communications and Networking Conference (WCNC), IEEE, 2022, pp. 1461–1466.
- [38] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, O. Pieter Abbeel, W. Zaremba, Hindsight experience replay, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [39] Y. Bengio, J. Louradour, R. Collobert, J. Weston, Curriculum learning, in: Proceedings of the 26th Annual International Conference on Machine Learning, 2009, pp. 41–48.
- [40] X. Shao, G. Sun, W. Yao, X. Li, O. Zhang, Fractional-order resolved acceleration control for free-floating space manipulator with system uncertainty, *Aerosp. Sci. Technol.* 118 (2021) 107041.
- [41] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, *arXiv preprint, arXiv:1707.06347*, 2017.
- [42] C. Berner, G. Brockman, B. Chan, V. Cheung, P. Debiak, C. Dennison, D. Farhi, Q. Fischer, S. Hashme, C. Hesse, et al., Dota 2 with large scale deep reinforcement learning, *arXiv preprint, arXiv:1912.06680*, 2019.
- [43] L. Engstrom, A. Ilyas, S. Santurkar, D. Tsipras, F. Janoos, L. Rudolph, A. Madry, Implementation matters in deep policy gradients: a case study on ppo and trpo, *arXiv preprint, arXiv:2005.12729*, 2020.
- [44] R. Chai, A. Tsourdos, A. Savvaris, S. Chai, Y. Xia, C.P. Chen, Six-dof spacecraft optimal trajectory planning and real-time attitude control: a deep neural network-based approach, *IEEE Trans. Neural Netw. Learn. Syst.* 31 (11) (2019) 5005–5013.
- [45] S. Wang, Y. Cao, X. Zheng, T. Zhang, A learning system for motion planning of free-float dual-arm space manipulator towards non-cooperative object, *Aerosp. Sci. Technol.* 131 (2022) 107980.
- [46] R. Chai, Y. Guo, Z. Zuo, K. Chen, H.-S. Shin, A. Tsourdos, Cooperative motion planning and control for aerial-ground autonomous systems: methods and applications, *Prog. Aerosp. Sci.* 146 (2024) 101005.
- [47] M. Wilde, C. Clark, M. Romano, Historical survey of kinematic and dynamic spacecraft simulators for laboratory experimentation of on-orbit proximity maneuvers, *Prog. Aerosp. Sci.* 110 (2019) 100552.